Schema Refinement & Normal Forms

- **SCHEMA REFINEMENT**
  - Introduction to schema refinement
  - Functional dependencies
  - Reasoning about FDs
- **NORMAL FORMS**
  - 1NF, 2NF, 3NF, BCNF
  - Properties of decompositions, normalization, schema refinement in database design
  - Other kinds of dependencies: 4NF, 5NF, DKNF
  - Case studies

The Evils of Redundancy

- Redundancy is at the root of several problems associated with relational schemas:
  - ❑redundant storage, insert/delete/update anomalies
- Integrity constraints, in particular functional dependencies, can be used to identify schemas with such problems and to suggest refinements.
- Main refinement technique: decomposition (replacing ABCD with, say, AB and BCD, or ACD and ABD).
- Decomposition should be used judiciously:
  - ❑Is there reason to decompose a relation?
  - ❑What problems (if any) does the decomposition cause?

INTRODUCTION TO SCHEMA REFINEMENT

Problems Caused by Redundancy

Storing the same information redundantly, that is, in more than one place within a database, can lead to several problems:

- Redundant storage: Some information is stored repeatedly.
- Update anomalies: If one copy of such repeated data is updated, an inconsistency is created unless all copies are similarly updated.
- Insertion anomalies: It may not be possible to store some information unless some other information is stored as well.
- Deletion anomalies: It may not be possible to delete some information without losing some other information as well.

INTRODUCTION TO SCHEMA REFINEMENT

Problems Caused by Redundancy (cont.)

Consider a relation obtained by translating a variant of the Hourly_Emps entity set

- Ex: Hourly_Emps(ssn, name, lot, rating, hourly wages, hours worked)
- The primary key for Hourly_Emps is ssn.
- In addition, suppose that the hourly wages attribute is determined by the rating attribute. That is, for a given rating value, there is only one permissible hourly wages value.
- This IC is an example of a functional dependency.
- It leads to possible redundancy in the relation Hourly_Emps

Use of Decomposition

Intuitively, redundancy arises when a relational schema forces an association between attributes that is not natural.

- Functional dependencies (ICs) can be used to identify such situations and to suggest refinements to the schema.
- The essential idea is that many problems arising from redundancy can be addressed by replacing a relation with a collection of smaller relations.
- Each of the smaller relations contains a subset of the attributes of the original relation.
- We refer to this process as decomposition of the larger relation into the smaller relations

Use of Decomposition (cont.)

We can deal with the redundancy in Hourly_Emps by decomposing it into two relations:

- Hourly_Emps2(ssn, name, lot, rating, hours worked)
- Wages(rating, hourly wages)

rating	*hourly wages*
8	10
5	7

Use of Decomposition (cont.)

ssn	*name*	*lot*	*rating*	*hours worked*
123-22-3666	Attishoo	48	8	40
231-31-5368	Smiley	22	8	30
131-24-3650	Smethurst	35	5	30
434-26-3751	Guldu	35	5	32
612-67-4134	Madayan	35	8	40

Problems related to Decomposition

Unless we are careful, decomposing a relation schema can create more problems than it solves.

Two important questions must be asked repeatedly:

1. Do we need to decompose a relation?
2. What problems (if any) does a given decomposition cause?

To help with the 1st question, several normal forms have been proposed for relations.

If a relation schema is in one of these normal forms, we know that certain kinds of problems cannot arise.

FUNCTIONAL DEPENDENCIES (FDs)

- A Functional Dependency (FD) X → Y (read as X determines Y) (X ⊆ R, Y ⊆ R) holds over relation R if, for every allowable instance r of R:
  - t1 ∈r, t2 ∈r, π_X(t1) = π_X(t2) implies π_Y(t1) = π_Y(t2)
  - i.e., given two tuples in r, if the X values agree, then the Y values must also agree. (X and Y are sets of attributes.)
- An FD is a statement about all allowable relations.
  - Must be identified based on semantics of application.
  - Given some allowable instance r1 of R, we can check if it violates some FD f, but we cannot tell if f holds over R!
- K is a candidate key for R means that K → R
  - However, K → R does not require K to be minimal!

FUNCTIONAL DEPENDENCIES (FDs) - Examples

Consider the schema:

Student (studName, rollNo, gender, dept, hostelName, roomNo)

- Since rollNo is a key, rollNo → {studName, gender, dept, hostelName, roomNo}
- Suppose that each student is given a hostel room exclusively, then

  hostelName, roomNo → rollNo
- Suppose boys and girls are accommodated in separate hostels, then

  hostelName → gender
- FDs are additional constraints that can be specified by designers

Trivial / Non - Trivial FDs

- An FD $X \rightarrow Y$ where $Y \subseteq X$

  called a trivial FD, it always holds good
- An FD $X \rightarrow Y$ where $Y \not\subseteq X$

  -non-trivial FD
- An FD $X \rightarrow Y$ where $X \cap Y = \emptyset$

  -completely non-trivial FD

FUNCTIONAL DEPENDENCIES (FDs) cont.

Constraints on Entity Set

- Consider relation obtained from Hourly_Emps:

  Hourly_Emps (ssn, name, lot, rating, hrly_wages, hrs_worked)
- Notation: We will denote this relation schema by listing the attributes: SNLRWH
  - This is really the set of attributes {S, N, L, R, W, H}.
  - Sometimes, we will refer to all attributes of a relation by using the relation name. (e.g., Hourly_Emps for SNLRWH)
- Some FDs on Hourly_Emps:
  - ssn is the key: S → SNLRWH
  - rating determines hrly_wages: R → W

Example (Contd.)

- Problems due to R → W :
  - Update anomaly: Can we change W in just the 1st tuple of SNLRWH?
  - Insertion anomaly: What if we want to insert an employee and don't know the hourly wage for his rating?
  - Deletion anomaly: If we delete all employees with rating 5, we lose the information about the wage for rating 5!

Wages

R	W
8	10
5	7

Hourly_Emps2

S	N	L	R	H
123-22-3666	Attishoo	48	8	40
231-31-5368	Smiley	22	8	30
131-24-3650	Smethurst	35	5	30
434-26-3751	Guldu	35	5	32
612-67-4134	Madayan	35	8	40

S	N	L	R	W	H
123-22-3666	Attishoo	48	8	10	40
231-31-5368	Smiley	22	8	10	30
131-24-3650	Smethurst	35	5	7	30
434-26-3751	Guldu	35	5	7	32
612-67-4134	Madayan	35	8	10	40

Constraints on a Relationship Set

- Suppose that we have entity sets Parts, Suppliers, and Departments, as well as a relationship set Contracts that involves all of them. We refer to the schema for Contracts as CQPSD. A contract with contract id
- C species that a supplier S will supply some quantity Q of a part P to a department D. We might have a policy that a department purchases at most one part from any given supplier.
- Thus, if there are several contracts between the same supplier and department, we know that the same part must be involved in all of them. This constraint is an FD, DS ! P.

Reasoning about Functional Dependencies (FDs)

- Given some FDs, we can usually infer additional FDs:
  - ssn → did, did → lot implies ssn → lot
- An FD f is implied by a set of FDs F if f holds whenever all FDs in F hold.
  - F^+ = closure of F is the set of all FDs that are implied by F.
- closure of F is the set of all FDs that are implied by F.

  Armstrong's Axioms (X, Y, Z are sets of attributes):
  - Reflexivity: If X -> Y, then Y -> X
  - Augmentation: If X -> Y, then XZ -> YZ for any Z
  - Transitivity: If X -> Y and Y -> Z, then X -> Z
- These are sound and complete inference rules for FDs!

Reasoning About FDs (Contd.)

Couple of additional rules (that follow from AA):

Union: If X → Y and X → Z, then X → YZ

Decomposition: If X → YZ, then X → Y and X →Z

Example: Contracts(cid, sid, jid, did, pid, qty, value), and:

C is the key: C → CSJDPQV

Project purchases each part using single contract: JP → C

Dept purchases at most one part from a supplier:SD → P

P → C, C → CSJDPQV imply JP → CSJDPQV

SD → P implies SDJ → JP

SDJ → JP, JP → CSJDPQV imply SDJ → CSJDPQV

Reasoning About FDs (Contd.)

- Computing the closure of a set of FDs can be expensive. (Size of closure is exponential in # attrs!)
- Typically, we just want to check if a given FD X → Y is in the closure of a set of FDs F. An efficient check:
  - Compute attribute closure of X (denoted by X^+) wrt F:

    Set of all attributes A such that X → A is in F^+

    There is a linear time algorithm to compute this.
  - Check if Y is in X^+
- Does F = {A → B, B → C, CD → E } imply A → E?

  i.e, is A → E in the closure F^+? Equivalently, is E in A^+?

Attribute Closure

- If we just want to check whether a given dependency, say, X → Y, is in the closure of a set F of FDs,
- we can do so efficiently without computing F^+. We 1st compute the attribute closure X^+ with respect to F, which is the set of attributes A such that X → A can be inferred using the Armstrong Axioms.
- The algorithm for computing the attribute closure of a set X of attributes is closure = X;

repeat until there is no change: {

if there is an FD U → V in F such that U subset of closure,

then

set closure = closure union of V

}

Database Normalization

- The main goal of Database Normalization is to restructure the logical data model of a database to:
  - Eliminate redundancy
  - Organize data efficiently
  - Reduce the potential for data anomalies.

Database Normalization Definitions

- How to take a raw collection of data and break it up into more logical units or tables, in order to reduce the occurrence of redundant data in the database. This process of reducing data redundancy is referred to as Normalization.
- Normalization is a body of rules addressing analysis and conversion of data structures into relations that exhibit more desirable properties of internal consistency, minimal redundancy and maximum stability.

Database Normalization Definitions

Normalization is the process by which attributes are grouped together to form a well-structured relation.

We focused on the characteristics of a good relation:

- Analyzing sample relations
- Identifying design flaws
- And learning how to eliminate them

This is called Normalizing a relation

Normalization is a process of decomposing relations to produce smaller, well-structured relations.

Normalization is a tool to validate and improve a logical design, so that it satisfies certain constraints that avoid unnecessary duplication of data.

Data Anomalies

- Data anomalies are inconsistencies in the data stored in a database as a result of an operation such as update, insertion, and/or deletion.
- One such scenario in which such inconsistencies may arise is a particular record stored in multiple locations and not all of the copies are updated
- We can prevent such anomalies by implementing 7 different level of normalization called Normal Forms (NF)

Brief History/Overview

- Database Normalization was first proposed by Edgar F. Codd.
- Codd defined the first three Normal Forms, which we'll look into, of the 7 known Normal Forms.
- In order to do normalization we must know what the requirements are for each of the three Normal Forms that we'll go over
- One of the key requirements to remember is that Normal Forms are progressive. That is, in order to have 3rd NF we must have 2nd NF and in order to have 2nd NF we must have 1st NF.

NORMAL FORMS

- Normal Form is a state of a relation that result by decomposing that relation for a good design to avoid redundancy.
- The Normal Forms defined in Relational database theory represent guidelines for record design.
- The design guidelines are meaningful even if one is not using a relational database system.
- We present the guidelines without referring to the concepts of the relational model in order to emphasize their generality, and also to make them easier to understand.

Steps in Normalization

- Normalization can be accomplished and understood in steps, and each step results in a Normal Form.
- The Normal Forms are ways of measuring the levels, or depth to which a database has been normalized.
- Normal Forms must be implemented in sequential order.

NORMAL FORMS

- First Normal Form (1NF)
  - Included in the definition of a relation
  - Table format; no repeating groups and Primary Key (PK) identified
- Second Normal Form (2NF)
  - 1NF and no partial dependencies
- Third Normal Form (3NF)
  - 2NF and no transitive dependencies
- Boyce-Codd Normal Form (BCNF)
  - Every determinant is a candidate key (special case of 3NF)

Defined in terms of functional dependencies

NORMAL FORMS

- Fourth Normal Form (4NF)
  - Defined using multivalued dependencies
  - 3NF and no independent multivalued dependencies
- Fifth Normal Form (5NF) or Project Join Normal Form (PJNF)
  - Defined using join dependencies

NORMAL FORMS

- The normal forms based on FDs are 1st normal form (1NF), second normal form (2NF), third normal form (3NF), and Boyce-Codd normal form (BCNF).
- These forms have increasingly restrictive requirements: Every relation in BCNF is also in 3NF, every relation in 3NF is also in 2NF, and every relation in 2NF is in 1NF.
- A relation is in first normal form if every field contains only atomic values, that is, not lists or sets.
- This requirement is implicit in our definition of the relational model.
- Although some of the newer database systems are relaxing this requirement
- 2NF is mainly of historical interest.
- 3NF and BCNF are important from a database design standpoint.

1st Normal Form: The Requirements

- The requirements to satisfy the 1st NF:
  - Each table has a primary key: minimal set of attributes which can uniquely identify a record
  - The values in each column of a table are atomic (No multi-value attributes allowed).
  - There are no repeating groups: two columns do not store similar information in the same table.
- 1st NF definition:

  1. A relation is in the First Normal Form, if it does not contain any repeating elements or groups.

  2. A relation is in the First Normal Form only if all underlying domains contain only atomic values.

1st Normal Form

- The objective of the First Normal Form is to divide the base data into logical units called tables.
- Once each table has been designed, a Primary Key is assigned to most or all tables.
- A Primary Key in a table is one or more columns that make every row of data in a table unique.
- “The First Normal Form deals with the shape of a record type”.
- Under the First Normal Form, all occurrences of a record type must contain the same no. of fields.

1st Normal Form

- First Normal Form excludes variable repeating fields and groups.
- This is not so much a design guideline as a matter of definition.
- Relational database theory doesn’t deal with records having a variable no. of fields.

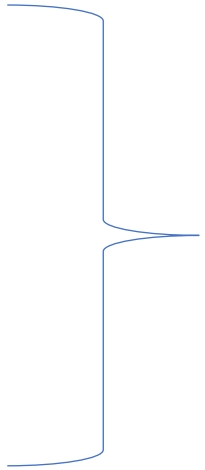
Drawback of First Normal Form:

- The main drawback of First Normal Form is redundancy of data.

1st Normal Form: Example - 1

- ORDER:

1. Order_No
2. Order_Date
3. Customer_No
4. Item_No
5. Item_Name
6. Qty_Ordered
7. Rate_Per_Unit
8. Item_Value



As many as items ordered in the ORDER

1st Normal Form: Example - 1

- In this example (ORDER) contains the repeating groups in item details. So it is not in the First Normal Form.
- To convert a relation into the First Normal Form, " remove all repeating (or multivalued) attributes to another (child) relation".
- We perform the operation on ORDER relation and arrive at the following two relations:

ORDER
1. Order_No (PK)
2. Order_Date
3. Customer_No

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Item_Name
4. Qty_Ordered
5. Rate_Per_Unit
6. Item_Value

1st Normal Form: Example – 1 (contd ..)

ORDER
1. Order_No (PK)
2. Order_Date
3. Customer_No

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Item_Name
4. Qty_Ordered
5. Rate_Per_Unit
6. Item_Value

- If we examine the relations ORDER and ORDER_ITEM, we find that there are no repeating elements or groups in both the relations and can therefore say that both these relations are in the First Normal Form.
- The First Normal Form is also called the Normalized Form (as against fully normalized).

1st Normal Form: Example - 2

- Un-normalized Students table:

Student#	AdvID	AdvName	AdvRoom	Class1	Class2
123	123A	James	555	102-8	104-9
124	123B	Smith	467	209-0	102-8

- Normalized Data

Student#	AdvID	AdvName	AdvRoom	Class#
123	123A	James	555	102-8
123	123A	James	555	104-9
124	123B	Smith	467	209-0
124	123B	Smith	467	102-8

1st Normal Form: Example - 3

- A table is considered to be in 1NF if all the fields contain only scalar values (as opposed to list of values).

  Example (Not 1NF)

ISBN	Title	AuName	AuPhone	PubName	PubPhone	Price
0-321-32132-1	Balloon	Sleepy, Snoopy, Grumpy	321-321-1111, 232-234-1234, 665-235-6532	Small House	714-000-0000	$34.00
0-55-123456-9	Main Street	Jones, Smith	123-333-3333, 654-223-3455	Small House	714-000-0000	$22.95
0-123-45678-0	Ulysses	Joyce	666-666-6666	Alpha Press	999-999-9999	$34.00
1-22-233700-0	Visual Basic	Roman	444-444-4444	Big House	123-456-7890	$25.00

AuName and AuPhone columns are not scalar

1NF - Decomposition

1. Place all items that appear in the repeating group in a new table
2. Designate a primary key for each new table produced.
3. Duplicate in the new table the primary key of the table from which the repeating group was extracted or vice versa.

Example (1NF)

ISBN	Title	PubName	PubPhone	Price
0-321-32132-1	Balloon	Small House	714-000-0000	$34.00
0-55-123456-9	Main Street	Small House	714-000-0000	$22.95
0-123-45678-0	Ulysses	Alpha Press	999-999-9999	$34.00
1-22-233700-0	Visual Basic	Big House	123-456-7890	$25.00

ISBN	AuName	AuPhone
0-321-32132-1	Sleepy	321-321-1111
0-321-32132-1	Snoopy	232-234-1234
0-321-32132-1	Grumpy	665-235-6532
0-55-123456-9	Jones	123-333-3333
0-55-123456-9	Smith	654-223-3455
0-123-45678-0	Joyce	666-666-6666
1-22-233700-0	Roman	444-444-4444

2nd Normal Form: The Requirements

- The requirements to satisfy the 2nd NF:
  - All requirements for 1st NF must be met.
  - Redundant data across multiple rows of a table must be moved to a separate table.
    - The resulting tables must be related to each other by use of foreign key.
- 2nd NF definition:
  - A relation is in the Second Normal Form if it is in the First Normal Form and all non-key attributes are fully functionally dependent on the primary key.
  - A relation is in 2NF if all the non-key attributes are dependant on all of the Key attributes.

2nd Normal Form

- 2nd NF is based on the concept of Full Functional dependency and removal of the partial functional dependency:
- Full FD:
  - A FD X → Y is a Full FD if removal of any attribute from X means that the dependency does not hold any more; i.e for any attribute $A \in X$ (X – {A}) does not Functionally determine Y
- Partial FD:
  - A FD X → Y is a Partial FD if some attribute $A \in X$ (X – {A}) does Functionally determine Y

2nd Normal Form (cont.)

- The conversion to Second Normal Form has taken place by removing attributes that are not dependent on the Full Primary Key attributes.
- A relation schema R is in 2NF if every non-prime attribute is fully functionally dependent on any key of R.
- Prime Attribute: An attribute that is part of some key
- Non-prime Attribute: An attribute that is not part of any key

2nd Normal Form: Example - 1

ORDER
1. Order_No (PK)
2. Order_Date
3. Customer_No

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Item_Name
4. Qty_Ordered
5. Rate_Per_Unit
6. Item_Value

- If we examine the relation ORDER_ITEM, we find that the Item_Name is not Fully Functionally dependent on the Full Primary Key(Order_No + Item_No) as it is Functionally dependent on a part of the Primary Key i.e Item_No.
- In otherwords, we do not need to know the Order_No to determine the Item_Name; we can determine it from Item_No alone.

2nd Normal Form: Example - 1 (cont.)

- The disadvantage of having such a relation is that if the name of an Item changes, it has to be changed in all the ORDER_ITEM rows where it occurs.
- To remove this disadvantage, we split the relation ORDER_ITEM into the following two relations

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Qty_Ordered
4. Rate_Per_Unit
5. Item_Value

ITEM
1. Item_No (PK)
2. Item_Name

Both these relations are now in the Second Normal Form (since we have assumed that the rate_per_unit cannot be derived from the Item_No.

1st Normal Form: Example

- Un-normalized Students table:

Student#	AdvID	AdvName	AdvRoom	Class1	Class2
123	123A	James	555	102-8	104-9
124	123B	Smith	467	209-0	102-8

- Normalized Data

Student#	AdvID	AdvName	AdvRoom	Class#
123	123A	James	555	102-8
123	123A	James	555	104-9
124	123B	Smith	467	209-0
124	123B	Smith	467	102-8

2nd Normal Form: Example - 2

- Students table

Student#	AdvID	AdvName	AdvRoom
123	123A	James	555
124	123B	Smith	467

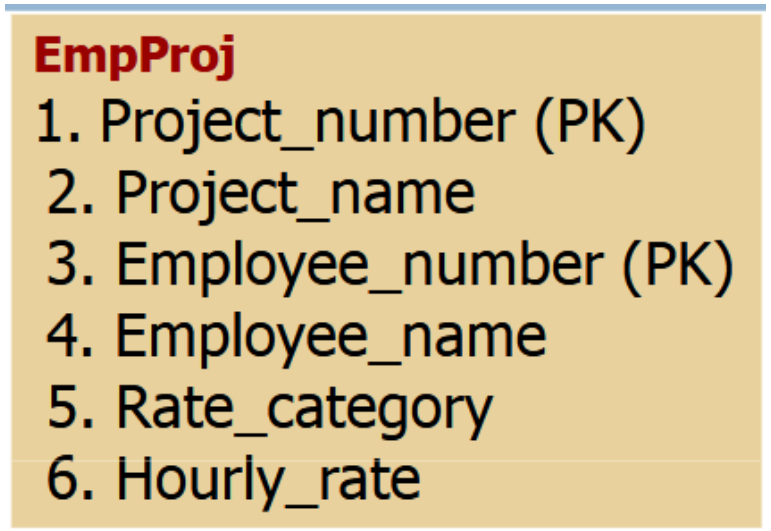
- Registration Table

Student#	Class#
123	102-8
123	104-9
124	209-0
124	102-8

Steps to convert a table to its 2nd Normal Form

- A table is in 2nd Normal Form if:
  - It is in 1st Normal Form
  - It includes no partial dependencies (where an attribute is dependent on only a part of a primary key)
- The steps to convert a table to its 2nd Normal Form:
  - Find and remove fields that are related to -- only part of the key.
  - Group the removed items in the another table.
  - Assign the new table with the key i.e. part of a whole composite key.

Steps to convert a table to its 2nd Normal Form Example - 3



- Going through all the fields reveals the following:
  - Project_name is only dependent on Project_number
  - Employee_name, Rate_category and Hourly_rate are dependent only on Employee_number

Steps to convert a table to its 2nd Normal Form Example - 3 (cont..)

EmpProj
1. Project_number (PK)
2. Project_name
3. Employee_number (PK)
4. Employee_name
5. Rate_category
6. Hourly_rate

- To convert the table into the Second Normal Form remove and place these fields in a separate table, with the key being that part of the original key they are dependent on.
- This leads to the following 3 tables:

EmpProj
1. Project_number(PK)
2. Employee_number(PK)

Emp
1. Employee_number (PK)
2. Employee_name
3. Rate_category
4. Hourly_rate

Proj
1. Project_number(PK)
2. Project_name

2nd Normal Form Example - 4

- 1)Book (authorName, title, authorAffiliation, ISBN, publisher, pubYear)

Keys: (authorName, title), ISBN

Not in 2NF as authorName, authorAffiliation (authorAffiliation is not fully functionally dependent on the first key)

2) Student (rollNo, name, dept, Gender, hostelName, roomNo, admitYear)

Keys: rollNo, (hostelName, roomNo)

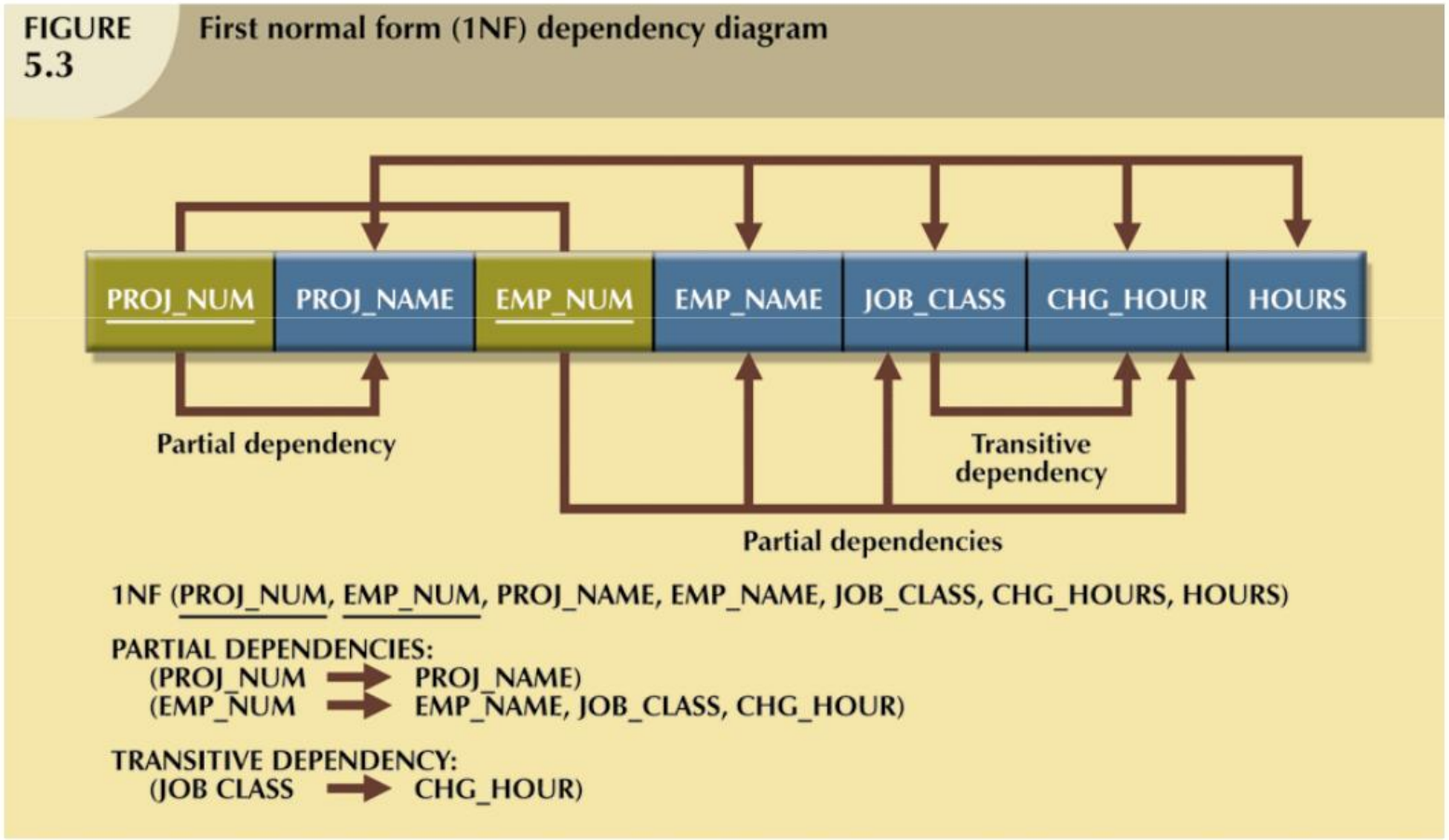
Not in 2NF as hostelName→Gender

student (rollNo, name, dept, hostelName, roomNo, admitYear)

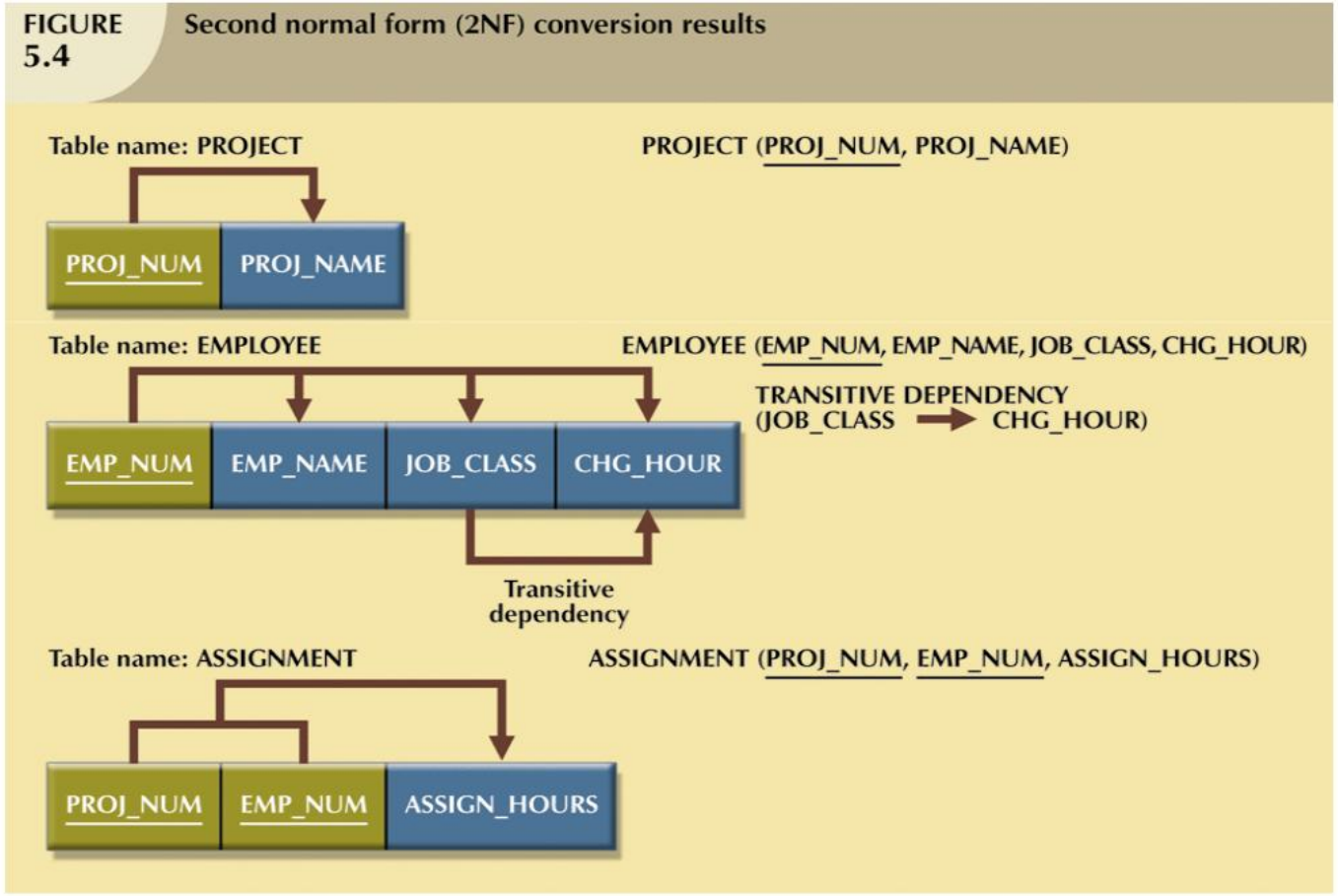
hostelDetail(hostelName, Gender)

-These are both in 2NF

2nd Normal Form Example - 5



2nd Normal Form Example - 5



3rd Normal Form: The Requirements

- The requirements to satisfy the 3rd NF:
  - All requirements for 2nd NF must be met.
  - Eliminate fields that do not depend on the primary key;
    - That is, any field that is dependent not only on the primary key but also on another field must be moved to another table.
- 3rd NF definition:

  1. A relation is in the 3NF if it is in the 2NF and every non-key attribute is non-transitively dependent on the primary key.

  2. A relation is in the 3NF if it is in the 2NF and every attribute is independent of all other non-key attributes.

3rd Normal Form: Example - 1

ORDER_ITEM

1. Order_No (PK)
2. Item_No (PK)
3. Qty_Ordered
4. Rate_Per_Unit
5. Item_Value

- We find that Item_Value can be derived from Qty_Ordered and Rate_Per_Unit (Item_Value = Qty_Ordered * Rate_Per_Unit) and hence each of these three attributes is dependent on the other two attributes.
- We reduce the relation into 3NF as follows:
- We have reduced the relation into its 3NF by removing attributes that depend on other non-key attributes.

3rd Normal Form: Example - 1

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Qty_Ordered
4. Rate_Per_Unit
5. Item_Value

Our original relation has now transformed to

ORDER
1. Order_No (PK)
2. Order_date
3. Customer_No

ORDER_ITEM
1. Order_No (PK)
2. Item_No (PK)
3. Qty_Ordered
4. Rate_Per_Unit

ITEM
1. Item_No (PK)
2. Item_Name

Transitive Dependencies

- Transitive dependency:
- An FD X →Y in a relation schema R for which there is a set Of attributes Z ⊆R such that
  - X →Z and Z →Y and Z is not a subset of any key of R

Ex: student (rollNo, name, dept, hostelName, roomNo, headDept)

Keys: rollNo, (hostelName, roomNo)

rollNo→dept; dept →headDept hold

So, rollNo→headDept a transitive dependency

Head of the dept D is stored redundantly in every tuple where D appears.

Relation is in 2NF but redundancy still exists.

3rd Normal Form: Example - 2

- Relation schema R is in 3NF if it is in 2NF and no non-prime attribute of R is transitively dependent on any key of R
- student (rollNo, name, dept, hostelname, roomNo, headDept)is not in 3NF
- Decompose:

  student (rollNo, name, dept, hostelName, roomNo)

  deptInfo(dept, headDept)

  both in 3NF

Redundancy in data storage -removed

Another definition of 3rd Normal Form

- Relation schema R is in 3NF if for any nontrivial FD
  - X →A either (i) X is a superkey or (ii) A is prime.
- Suppose some R violates the above definition

  ⇒ There is an FD X →A for which both (i) and (ii) are false

  ⇒ X is not a superkey and A is non-prime attribute
- Two cases arise:

  1) X is contained in a key –A is not fully functionally dependent on this key

  -violation of 2NF condition

  2) X is not contained in a key

  K →X, X →A is a case of transitive dependency (K –any key of R)

Another definition of 3rd Normal Form

- A Relation schema R is in 3NF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $X \rightarrow Y$, where $X \subseteq R$ and $Y \subseteq R$, atleast one of the following holds:

  (i) $X \rightarrow Y$ is a trivial functional dependency

  (ii) X is a Superkey for R

  (iii) Each attribute A in $Y - X$ is contained in a Candidate key for R

3rd Normal Form: Example - 3

- gradeInfo(rollNo, studName, course, grade)
- Keys: (rollNo, course), (studName, course)
- Suppose the following FDs hold:
  1) rollNo, course →grade
  2) studName, course →grade
  3) rollNo→studName
  4) studName→rollNo
- For 1,2 lhs is a key.
- For 3,4 rhs is prime
  -So gradeInfo is in 3NF
- But studName is stored redundantly along with every course being done by the student

3rd Normal Form: Example - 4

- Students table:

Student#	AdvID	AdvName	AdvRoom
123	123A	James	555
124	123B	Smith	467

- Student table:

Student#	AdvID
123	123A
124	123B

Advisor table:

AdvID	AdvName	AdvRoom
123A	James	555
123B	Smith	467

3rd Normal Form: Example - 4 Cont.

- Students table:

Student#	AdvID
123	123A
124	123B

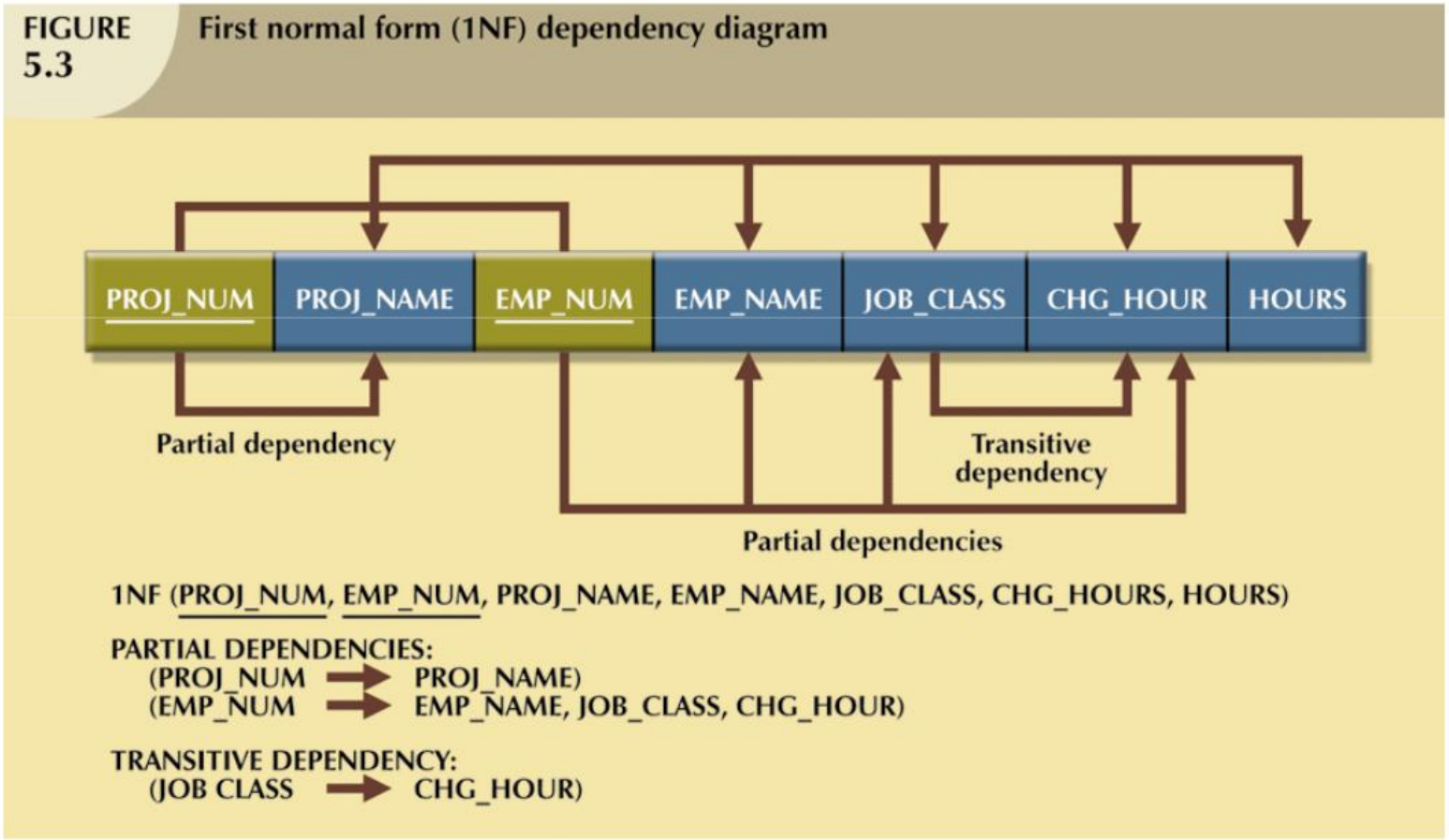
- Student table:

Student#	Class#
123	102-8
123	104-9
124	209-0
124	102-8

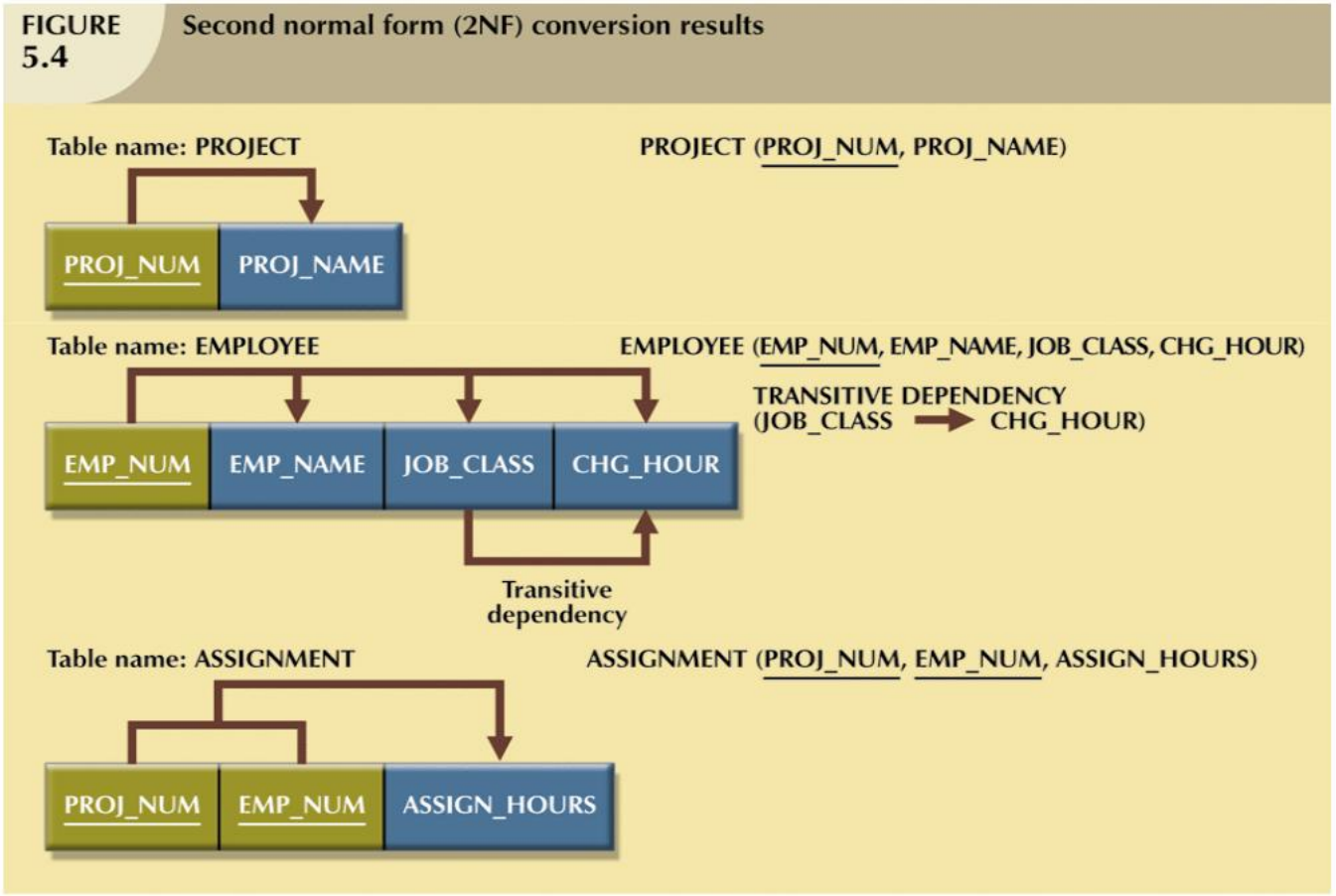
Advisor table:

AdvID	AdvName	AdvRoom
123A	James	555
123B	Smith	467

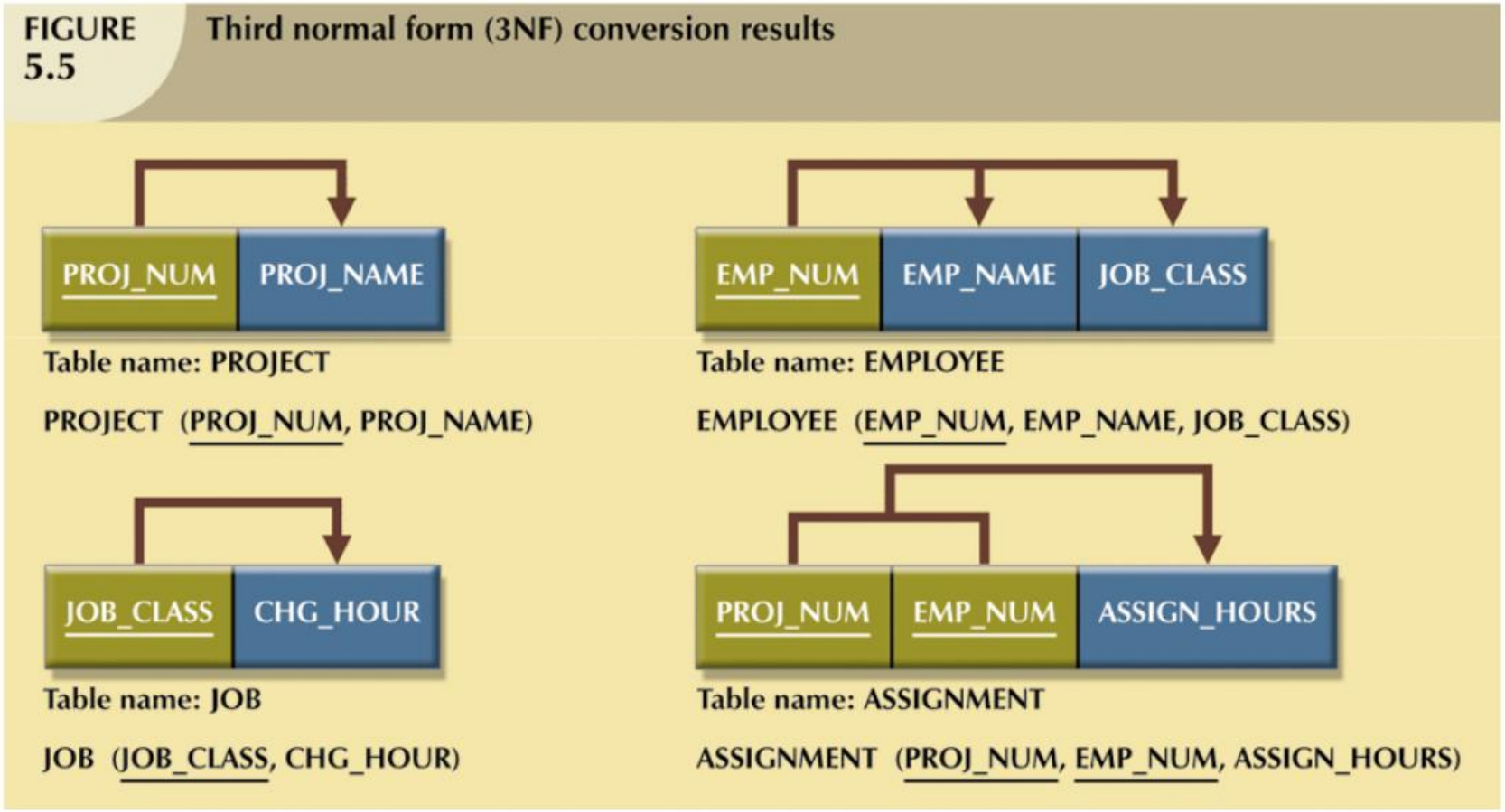
1st Normal Form Example



2nd Normal Form Example



3rd Normal Form
Example - 5



3rd Normal Form
Example - 6

EmpProj
1. Project_number (PK)
2. Project_name
3. Employee_number (PK)
4. Employee_name
5. Rate_category
6. Hourly_rate

- To convert the table into the Second Normal Form remove and place these fields in a separate table, with the key being that part of the original key they are dependent on.

This leads to the following 3 tables in 2NF:

EmpProj
1. Project_number(PK)
2. Employee_number(PK)

Emp
1. Employee_number (PK)
2. Employee_name
3. Rate_category
4. Hourly_rate

Proj
1. Project_number(PK)
2. Project_name

3rd Normal Form
Example - 6

- Going through all the fields reveals the following:
  - Employee table is the only one with more than one non-key attribute
  - Employee name is not dependent on either Rate_category or Hourly_Rate
  - Hourly_Rate is dependent on Rate_Category
  - To convert the table into 3NF remove and place these fields in a separate table, with the attribute it was dependent on as key, as follows:
  - This leads to the following 4 tables:

EmpProj
1. Project_number(PK)
2. Employee_number(PK)

Rate
1. Rate_category (PK)
2. Hourly_rate

Emp
1. Employee_number (PK)
2. Employee_name
3. Rate_category

Proj
1. Project_number(PK)
2. Project_name

BCNF Requirements

To satisfy BCNF:

- Must be in **3rd Normal Form (3NF)**.
- For every functional dependency **X → Y**, **X must be a super key**.

BCNF - Example

- EmployeeDepartment Table

EmpID	DeptName	Manager
1	IT	John
2	IT	John
3	HR	Sara

Issue: DeptName → Manager (DeptName uniquely determines Manager)

Problem: DeptName is not a key, but it determines Manager.

BCNF - After Normalization

Department Table

DeptName	Manager
IT	John
HR	Sara

Employee Table

EmpID	DeptName
1	IT
2	IT
3	HR

Now DeptName is properly separated.

4NF - Requirements

- **To satisfy 4NF:**
  - Must be in **BCNF**.
  - No **multivalued dependencies** unless they are trivial.

4NF - Example

StudentID	Hobby	Skill
1	Painting	Dancing
1	Reading	Singing

Issue:

- StudentID → Hobby (multivalued dependency)
- StudentID → Skill (multivalued dependency)

4NF - After Normalization

- StudentHobby Table

StudentID	Hobby
1	Reading
1	Painting

StudentSkill Table

StudentID	Skill
1	Singing
1	Dancing

Multivalued dependencies are separated.

5NF - Prerequisites

To satisfy 5NF:

- Must be in **4NF**.
- No **join dependency** that is non-trivial.

5NF - Example

- SupplierPartsProject Table

Supplier	Part	Project
S1	P1	J1
S1	P1	J2

Issue:

•Can break into three separate tables: Supplier-Part, Part-Project, Supplier-Project.

5NF - After Normalization

SupplierPart Table

Supplier	Part
S1	P1

PartProject Table

Part	Project
P1	J1
P1	J2

SupplierProject Table

Supplier	Project
S1	J1
S1	J2

Lossless join achieved!

- THANK YOU